1.Insertion at Beginning

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){

    struct node *first,*second,*third,*newnode;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    newnode = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;
    newnode->data = 5;

    head = first;
```

```
list: 5 10 20 30

=== Code Execution Successful ===
```

main.c

Share Run

Output

```c
    head = first;

    first->prev = NULL;
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;

    newnode->prev = NULL;
    newnode->next = head;
    head->prev = newnode;
    head = newnode;
}

void display(){

    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
```

```
list: 5 10 20 30

=== Code Execution Successful ===
```

```c

    newnode->prev = NULL;
    newnode->next = head;
    head->prev = newnode;
    head = newnode;
}

void display(){

    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    printf("list: ");
    display();

    return 0;
}
```

```
list: 5 10 20 30

=== Code Execution Successful ===
```

2.Insertiom at End

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){

    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;

    first->prev = NULL;
```

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

```c

    head = first;

    first->prev = NULL;
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void insertend(){

    struct node *temp,*newnode;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 40;

    temp = head;

    while(temp->next != NULL){
        temp = temp->next;
    }

```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

```c
    temp->next = newnode;
    newnode->prev = temp;
    newnode->next = NULL;
}

void display(){

    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    insertend();

    printf("List: ");
    display();

    return 0;
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

3.Insertion at position

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){

    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;

    first->prev = NULL;
```

```
List: 10 15 20 30

=== Code Execution Successful ===
```

```c
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void insertpos(int pos){

    struct node *temp,*newnode;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 15;

    temp = head;

    for(i=1;i<pos-1;i++){
        temp = temp->next;
    }

    newnode->next = temp->next;
    newnode->prev = temp;
```

```
List: 10 15 20 30

=== Code Execution Successful ===
```

```c

    temp->next->prev = newnode;
    temp->next = newnode;
}

void display(){

    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    insertpos(2);

    printf("List: ");
    display();

    return 0;
}
```

```
List: 10 15 20 30

=== Code Execution Successful ===
```

4.Deletion at Beginning

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){

    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;

    first->prev = NULL;
```

```
List: 20 30

=== Code Execution Successful ===
```

```c
    first->prev = NULL;
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void deletebegin(){

    struct node *temp;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    temp = head;
    head = head->next;

    if(head != NULL){
        head->prev = NULL;
    }

```

```
List: 20 30

=== Code Execution Successful ===
```

```c
    }

    free(temp);
}

void display(){

    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    deletebegin();

    printf("List: ");
    display();

    return 0;
}
```

```
List: 20 30

=== Code Execution Successful ===
```

5.Deletion at End

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){

    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;

    first->prev = NULL;
```

```
List: 10 20

=== Code Execution Successful ==
```

```c
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void deleteend(){

    struct node *temp;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    temp = head;

    while(temp->next != NULL){
        temp = temp->next;
    }

    temp->prev->next = NULL;
```

```
List: 10 20

=== Code Execution Successful ===
```

```c
    temp->prev->next = NULL;

    free(temp);
}

void display(){

    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    deleteend();

    printf("List: ");
    display();

    return 0;
}
```

```
List: 10 20

=== Code Execution Successful ===
```

6.Deletion at position

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning(){

    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;

    first->prev = NULL;
```

```
List: 10 30

=== Code Execution Successful ===
```

```c
    first->next = second;

    second->prev = first;
    second->next = third;

    third->prev = second;
    third->next = NULL;
}

void deletepos(int pos){

    struct node *temp;
    int i;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    temp = head;

    for(i=1;i<pos;i++){
        temp = temp->next;
    }

    temp->prev->next = temp->next;
```

```
List: 10 30

=== Code Executi
```

```c

    temp->prev->next = temp->next;

    if(temp->next != NULL){
        temp->next->prev = temp->prev;
    }

    free(temp);
}

void display(){

    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    deletepos(2);

```

```
List: 10 30

=== Code Execution Successful ===
```

```c
    }

    free(temp);
}

void display(){

    struct node *temp = head;

    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    deletepos(2);

    printf("List: ");
    display();

    return 0;
}
```

```
List: 10 30

=== Code Execution Successful ===
```